



Correct-by-construction requirement decomposition

Minghui Sun, Georgios Bakirtzis, Hassan Jafarzadeh, Cody Fleming

► To cite this version:

Minghui Sun, Georgios Bakirtzis, Hassan Jafarzadeh, Cody Fleming. Correct-by-construction requirement decomposition. *Software and Systems Modeling*, 2025, pp.1-16. 10.1007/s10270-025-01291-4 . hal-05110632

HAL Id: hal-05110632

<https://hal.science/hal-05110632v1>

Submitted on 13 Jun 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Correct-by-construction requirement decomposition

Minghui Sun^{1*}, Georgios Bakirtzis^{2,3}, Hassan Jafarzadeh⁴,
Cody Fleming⁵

^{1*}Nanjing University of Information Science and Technology.

²Télécom Paris.

³Institut Polytechnique de Paris.

⁴Aptiv.

⁵Iowa State University.

*Corresponding author(s). E-mail(s): minghui_sun@nuist.edu.cn;

Abstract

In systems engineering, accurately decomposing requirements is crucial for creating well-defined and manageable system components, particularly in safety-critical domains. Despite the critical need, rigorous, top-down methodologies for effectively breaking down complex requirements into precise, actionable sub-requirements are scarce, especially compared to the wealth of bottom-up verification techniques. Addressing this gap, we introduce a formal decomposition for contract-based design that guarantees the correctness of decomposed requirements if specific conditions are met. Our (semi-)automated methodology augments contract-based design with reachability analysis and constraint programming to systematically identify, verify, and validate sub-requirements representable by continuous bounded sets—continuous relations between real-valued inputs and outputs. We demonstrate the efficacy and practicality of a correct-by-construction approach through a comprehensive case study on a cruise control system, highlighting how our methodology improves the interpretability, tractability, and verifiability of system requirements.

1 Introduction

Requirement decomposition is integral in systems engineering and requirements engineering (RE), involving breaking down top-level requirements into lower-level, actionable sub-requirements. This practice is fundamental to creating functional and feasible systems, guiding the development from conceptual design to tangible products. Despite being a core process recommended by numerous systems engineering guidelines such as NASA ([National](#)

Aeronautics and Space Administration and Space Science Library, 2010), EIA632 (ANSI/EIA 632, 2003), and FAA (DOT/FAA/AR-08/32, 2009), the field continues to grapple with a significant challenge: the lack of standardized, effective formalized RE processes (Kirkman, 1998; Braun et al, 2014). Current practices, predominantly manual and reliant on the expertise of designers, are time-consuming and error-prone. In particular, manual approaches to RE not only introduce a high probability of errors but also tend to conceal these errors until the later stages of development, like integration and testing. These more costly and complex stages potentially jeopardize the project’s timeline and success.

Therefore, formal verification has been increasingly adopted as a more rigorous analytical technique to enhance the quality of requirement decomposition. Design solutions are formally verified through a computational model, allowing for a detailed and rigorous analysis to confirm that a design meets its specified requirements (Baier and Katoen, 2008). This approach provides a layer of assurance beyond traditional methods, bolstering confidence in the correctness of design solutions (Ouimet and Lundqvist, 2007). Despite these advancements, formal verification primarily addresses the validation of final designs and falls short in ensuring the initial accuracy and correctness of sub-requirements.

Incorrectly decomposed requirements can lead to significant issues in system *composability*, *realizability*, and *reusability*. The downstream impacts of such deficiencies are well-documented, with instances like the Boeing 787’s delay due to supply chain complications (Greising and Johnsson, 2007) and the catastrophic Ariane 5 crash attributable to software reuse errors (Le Lann, 1997) highlighting the severe consequences of inadequate requirement decomposition. Current literature and practices in RE often fail to provide a holistic solution, focusing more on management and procedural aspects rather than the technical depths of requirements specification, technical design, verification, and validation (Wagner et al, 2019; Penzenstadler, 2011). This gap underscores the necessity for a systematic approach that addresses the entire spectrum of requirement decomposition, from the initial definition to the final validation of sub-requirements.

Contract-based design (CBD) emerges as a structured approach to system development, directly addressing many of the challenges inherent in requirements decomposition (Abbasipour, 2018; Melancon, 2008; Sidky et al, 2002). CBD’s core involves crafting formal agreements or contracts that specify the expected behavior and interactions between system components, providing a precise, enforceable, and verifiable foundation from the outset. This method promotes clarity, accuracy, and rigor in the early stages of requirement decomposition, contributing to a more disciplined and reliable engineering process. However, the efficacy of CBD is highly dependent on the accurate initial definition of contracts, which requires a comprehensive understanding of the system and its environment. Misinterpretations or oversights early on can lead to systemic errors, especially in complex systems characterized by high interconnectivity or evolving requirements.

In response to these challenges, this paper puts forth a semi-automated methodology that uses and combines top-down requirements decomposition processes from systems engineering and bottom-up verification techniques from formal methods, such that the decomposed requirements are correct by construction insofar as the requirements can be represented as continuous bounded sets. We focus on refining the initial formulation of contracts, ensuring they are comprehensive and adaptable to accommodate changes and design complexities. This targeted improvement reduces errors, enhances system composability, and ensures robust and

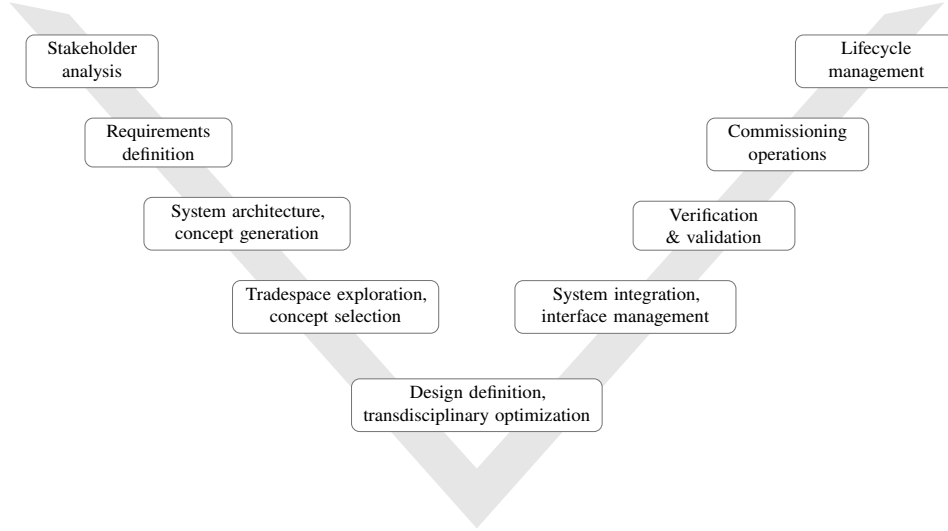


Fig. 1 Requirements engineering can be enhanced through the application of formal methods, typically employed on the right side of the V model, to ensure precision and accuracy (adapted from [ARP4754B \(2023\)](#)).

aligned decomposed requirements. Systematically generating sub-requirements through formal methods leads to correct-by-construction decomposition that minimizes the costly troubleshooting of system integration and validation at the right side of the V model (fig. 1). In particular, the correct-by-construction requirement decomposition systematically decomposes high-level contracts into implementable sub-requirements, coupled with semi-automated subcontract instantiation and optimization strategies to balance design robustness with physical constraints.

2 Preliminaries

Throughout this paper, we use specific notations to maintain clarity and consistency:

- Superscript numbers indicate contract identifiers.
- Subscript numbers denote variable identifiers.
- Superscript $*$ represents implementation specifics.
- Subscript R denotes refinement aspects.
- Over bar $\bar{}$ is the upper bound of a set.
- Under bar $\underline{}$ is the lower bound of a set.
- Tilde $\sim$ denotes the set of acceptable output values from the range of a function.
- The transpose of a matrix is denoted by superscript $^\top$.
- Variables f denote relations and variables g denote functions.

We generally view a system as an abstract entity, defined by its input-output relationships ([Iber et al, 2015](#)). This abstraction aligns with numerous theoretical frameworks and is compatible with various existing theories and tools, making it a practical model for real-world systems ([Mesarovic et al, 2000](#); [Kusiak and Larson, 1995](#)).

Sets, relations, and functions

For sets A and B , we define the following operations.

- Cartesian product: $A \times B = \{(a, b) | a \in A, b \in B\}$
- Power set: $\mathcal{P}(A) = \{S | S \subseteq A\}$

Let X and Y be sets. A relation f between sets X and Y is a subset of their Cartesian product: $f : X \rightarrow X \times Y$. A function $g : X \rightarrow Y$ maps each element $x \in X$ to a unique element $y \in Y$. In words, a *relation* is a pairing between elements from two sets, while a *function* is a relation with the extra property that each input mapping to exactly one output.

Assume-guarantee contracts

Assume-guarantee contracts provide a formal framework for specifying and reasoning about component-based systems (Benveniste et al, 2012a). Let X be a set of input values and Y be a set of output values for a component. An assume-guarantee contract C is, then, defined as a pair (A, G) , where:

- Assumption (A): $A \subseteq X$ represents the subset of inputs that are considered acceptable and expected by the environment.
- Guarantee (G): $G \subseteq X \times Y$ is a relation specifying valid input-output pairs that must hold true for the component to fulfill its contract.

Assume-guarantee contracts must adhere to two conditions.

C1 Acceptance of inputs: The component accepts all inputs within the assumption set A .

C2 Production of outputs: For every input $x \in A$, there exists an output y such that $(x, y) \in G$.

Whenever we discuss a *contract*, we subsequently mean a contract of this type within the formal framework.

System abstraction

In our framework, a system is abstracted as a relation from an input vector to an output vector, with all variables bounded within defined sets:

- Input vector: $x = (x_1, \dots, x_m)^T \in \mathbb{R}^m$
- Output vector: $y = (y_1, \dots, y_n)^T \in \mathbb{R}^n$
- Input variables are bounded: $x_i \in X_i$
- Output variables are bounded: $y_j \in Y_j$

Scalability of CBD

CBD has shown promising results in handling systems of moderate complexity. The hierarchical nature of contracts facilitates decomposition of large systems into manageable subsystems, allowing for scalable analysis and verification. Advanced techniques in constraint satisfaction problem (CSP) solving, such as problem decomposition and efficient constraint propagation algorithms, have significantly improved the ability to handle larger problem instances. Synthesizing a controller from contract specification has complexity of time polynomial, but careful choice of specifications can reduce compute time in practice (Filippidis and Murray, 2018).

3 Correct-by-construction requirement decomposition

In this section, we define the theoretical instantiation of correct-by-construction requirement decomposition. In particular, we define the different types of contracts for mapping top-down design to bottom-up verification methods, instantiate refinement operators for requirement decomposition to implementation, and construct bounded sets for the resulting formal methods.

3.1 Contracts for requirement decomposition

To expand the input-output relationship through a step-wise hierarchical refinement in the context of RE, we adopt assume-guarantee contracts (Ye et al, 2020) and augment their construction with the following definitions.

We assume that a decomposition exists for any given high-level contract, though we recognize this assumption carries limitations that merit discussion. Specifically, constraints within the intermediate representation domain could restrict the fidelity or expressiveness of certain decompositions, potentially limiting applicability across diverse contract types. Additionally, the depth of decomposition may impact both computational feasibility and interpretability, as deeper decompositions might introduce complexity that challenges the practical use of our method. Despite these limitations, this assumption is reasonable within our chosen framework, as it enables a systematic approach to contract analysis and facilitates manageable decomposition for the vast majority of scenarios relevant to our study.

Relational contract

A relational contract defines a desired relationship between each input value, X , and a set of acceptable output values, Y . Conceptually equivalent to a requirement in system design, it often holds a general form at higher levels to accommodate lower-level uncertainties. A relational contract, formally represented as $f : X \rightarrow X \times Y$, ensures all inputs $x \in X$ lead to acceptable outputs $y \in Y$. In the context of a relational contract, f is a *relation* that maps each input from the set X to a pair consisting of that input and an acceptable output from the set Y , effectively defining the set of allowable input-output behaviors for the system.

Functional contract

Adding structure to the relational contract, a functional contract includes uncertain parameters, denoted by P , within a functional framework, representing the intended behavior of a design solution. It is formally defined as $g : X \times P \rightarrow X \times Y$, with $g(x, p)$ being a function of the known or intended mechanism, and p denoting uncertainty. The uncertain parameters $p \in P$ typically represent aspects of the system or its environment that are not fully known or may vary within a specified range and are bounded within defined sets, allowing the contract to account for a range of possible system behaviors while maintaining the overall structure of the intended design solution.

Implementation

Focusing on deterministic behavior, an implementation is a specific realization of the system's input-output relationship. It is denoted as $y = f^*(x)$, where f^* is the mathematical function embodying the given system behavior through the (de)composition of contracts.

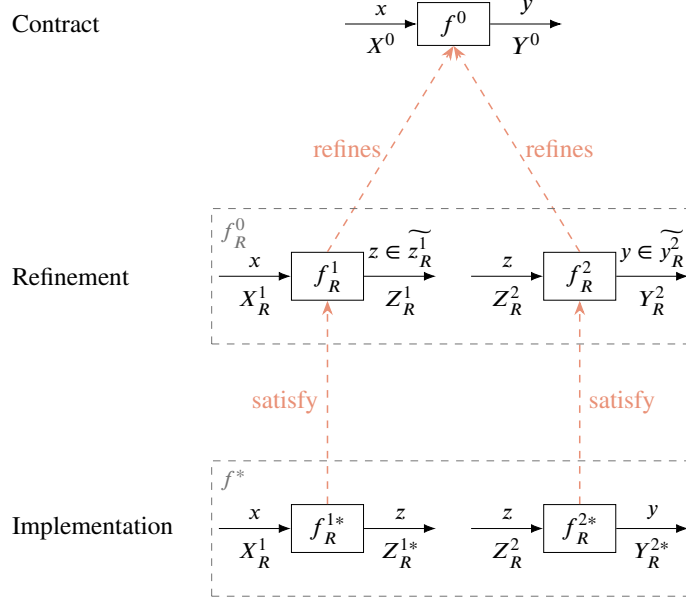


Fig. 2 Refinement by decomposition takes a high-level contract and reduces it to a candidate implementation.

Refinement

Refinement is the process of narrowing the range of acceptable values for given inputs, progressively moving from a more abstract contract towards a more concrete, implementable system specification. This process typically involves transitioning from a relational contract through increasingly specific functional contracts, ultimately leading to a deterministic system description. Each refinement step reduces the overall uncertainty in the contract, allowing for a more precise and implementable system design. Formally, refinement ensures that outputs acceptable in the refined contract are also acceptable in the original, while potentially reducing the set of acceptable outputs and/or expanding the set of allowed inputs. This maintains the contract's consistency while increasing its specificity.

Properties of input-output models

To ensure the integrity and functionality of systems designed using CBD, we rely on the existence of several formal properties of input-output abstractions.

- **Satisfiability:** An implementation satisfies the contract if all allowable inputs lead to acceptable outputs defined by the contract. This property ensures that the system's performance aligns with the defined requirements.
- **Composability:** This property addresses the interaction between system components. For two contracts to be composable, the output values of one must be acceptable as input values of the other, ensuring seamless system functionality.
- **Reachability:** Defined within functional contracts, reachability ensures that for any given input, the set of achievable outputs is included in the set of acceptable outputs defined by the

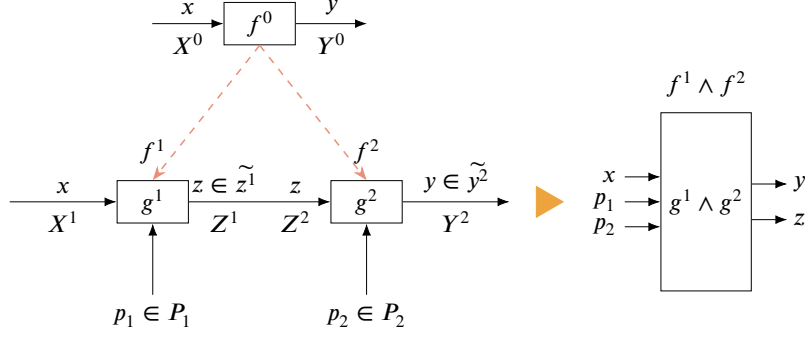


Fig. 3 Primary refinement decomposes top-level contracts and joins them in one representation.

contract. This property is crucial for ensuring the system can achieve the desired outcomes under various conditions.

- **Realizability:** Realizability pertains to the practicality of constructing devices that satisfy the contract's implementation. A contract is realizable if the supplier can produce devices capable of accepting all inputs and producing all outputs defined by the contract.

Requirement decomposition involves refining the system's high-level requirements into more specific, implementable sub-requirements that, when combined, satisfy the top-level requirement. Figure 2 illustrates a simplified view of this process, wherein a given contract f^0 is decomposed into subcontracts f_R^1 and f_R^2 , which are then independently refined into f_R^{1*} and f_R^{2*} —this procedure of contract definition and refinement can happen inductively for any number of contracts. Different suppliers subsequently provide devices that meet these refined subcontracts.

The decomposition process consists of three primary tasks:

- **Primary refinement:** f^1 and f^2 collectively refine the higher-level contract f^0 .
- **Secondary refinement:** The independently refined contracts, f_R^1 and f_R^2 , collectively refine f^0 .
- **Implementation:** Implementations f_R^{1*} and f_R^{2*} satisfy f_R^1 and f_R^2 respectively, and together satisfy f^0 .

Primary refinement entails that level 1 subcontracts, f^1 and f^2 , as a whole must refine the initial contract f^0 (fig. 3). Refinement on the reachable sets; that is, all possible states a system can reach, denoted by RA , must satisfy two properties.

P1 The subcontracts as a collective must satisfy *refinement*:

$$\forall x \in X^0 : RA_y \subseteq \tilde{y}^0, \\ X^0 \subseteq X^1, Y^2 \subseteq Y^0.$$

P2 The subcontracts must *encompass the reachable sets* of their respective variables:

$$RA_Y \subseteq Y^2, RA_Z \subseteq Z^1, \text{ and } RA_Z \subseteq Z^2.$$

Suppose the subcontracts of the primary refinement are composable and refined respectively by lower-level contracts. In that case, the lower-level contracts are composable and can refine the contract. Additionally, if the refined subcontracts of a given contract are composable and satisfied respectively by their implementations, then the implementations can satisfy the given contract as a whole. We deduce that if the subcontracts at all levels are composable and can refine their respective higher-level contracts, the resulting implementations are assured of satisfying the top-level contract. Therefore, for the sub-requirements to be *correct*, they must be admissible, composable contracts and refine the higher-level contract.

3.2 Semi-automated subcontract instantiation

This subsection discusses the initial step of defining a design solution for requirement decomposition (Komoto and Tomiyama, 2011). Architectural decomposition is typically a manual process, relying on the designer's understanding of the desired system; therefore, we assume it exists (Benveniste et al, 2015b).

Consider an example design solution (fig. 4) where a given contract f is decomposed into two subcontracts: f^1 and f^2 . The structure of the design solution includes functions f^1 and f^2 defined as $(x_3, x_4)^\top = g^1(x_1, x_2, p_1)$ and $x_2 = g^2(x_3, x_4, p_2)$ respectively, where $p_1 \in P_1$ and $p_2 \in P_2$. We define X_i^k as the bounded set for variable x_j of subcontract f^k .

To ensure that the design solution refines the original contract f , it must meet specific conditions, such as:

$$\forall x_1 \in X_1^0 : RA_{x_2} \subseteq \tilde{x}_2^0 \quad (1)$$

$$RA_{x_2} \subseteq X_2^0 \text{ and } X_1^0 \subseteq X_1^1 \quad (2)$$

These conditions, (1) and (2), are integral in deciding the design solution and the refined input sets. The applicable condition and verification may vary depending on whether the contract is relational or functional. It is also essential to distinguish between inputs and parameters during the refinement process, as their roles and behaviors differ significantly. The example provided is for illustration purposes, demonstrating our methodology's applicability to various architectural designs.

3.3 Continuous bounds

The lower bound: reachability analysis

Subcontracts serve as the blueprint for developing devices that, once integrated, support the system's operation. Reachable sets are used as the lower bound to ensure that the subcontracts encompass all possible operational conditions. Mathematically, this relationship is defined as:

$$\text{if } X_i^k \neq \emptyset, \text{ then } RA_{x_i} \subseteq X_i^k \quad (3)$$

Reachable sets are typically calculated through some form of reachability analysis. The reachable set is defined for a given initial state $x(0) \in \mathbb{R}^n$, system dynamics g , time-varying external control $u(t) \in \mathbb{R}^m$, input trajectory $u(\cdot)$, and parameter vector $p \in \mathbb{R}^l$. The continuous reachable set at time t is defined for a set of initial states $X(0)$, a set of uncertain time-varying

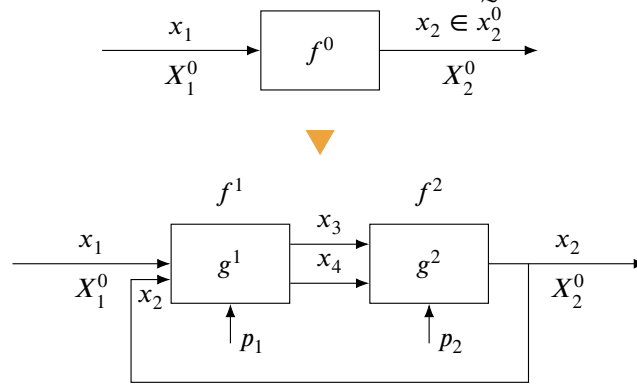


Fig. 4 Decomposition produces a potential design solution with associated behavioral guarantees.

external controls $U(t)$, and a set of uncertain but fixed parameter values P , as:

$$RA_x = \{g(x(0), u(\cdot), p) | x(0) \in X(0), u(t) \in U(t), p \in P\},$$

where $RA_x \in \mathbb{R}^n$ and RA_{x_i} is the reachable set of x_i .

The upper bound: constraint programming

While designers might aspire to build the most capable system possible, the practical limitations of device suppliers set the upper bound for subcontracts. Typically, subcontracts such as f^1 and f^2 are realized by different suppliers. Let S_i^k be the realizable set of variable x_i for subcontract f^k . Intuitively, suppliers can't build devices that exceed their capabilities. Mathematically, this relationship is defined as:

$$X_i^k \subseteq S_i^k$$

Realizable sets must be consistent and conflict-free. For example, if $S_3^1 \cap S_3^2 = \emptyset$, it indicates incompatibility between the devices for f^1 and f^2 . On the other hand, if $S_3^1 \cap S_3^2 \neq \emptyset$, the shared set becomes the new realizable set, affecting subsequent refinements.

The consistency among the realizable sets forms a CSP as defined by a set of variables, domains for each variable, and a set of constraints. This paper constructs a CSP following the general formulation and uses the constraint propagation algorithm to iteratively reduce the domain of variables until no further contraction is possible.

As a result, the contracted domain for each variable, denoted by RE_{x_i} , forms the upper bounds for the subcontracts:

$$\text{if } X_i^k \neq \emptyset, \text{ then } X_i^k \subseteq RE_{x_i}. \quad (4)$$

3.4 Trade-off study: optimization

Optimization of subcontracts involves balancing the system design's robustness with physical constraints, adhering to established correctness criteria. This process requires managing the tension between different subcontracts, especially when they share variables or interfaces.

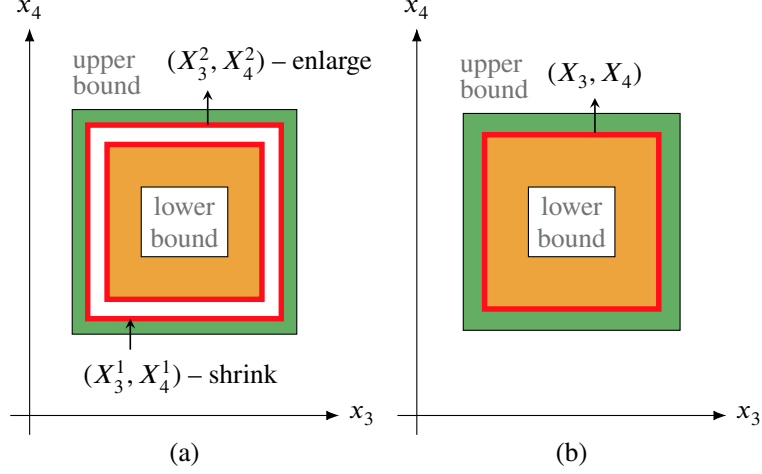


Fig. 5 The inherent tension between subcontracts.

Given a relational contract f , the optimization aims to define subcontracts f^1 and f^2 such that they are both correct individually and collectively satisfy the top-level requirement. We begin by establishing the constraints that potentially govern the functional subcontracts fig. 5:

$$\begin{aligned}
 g^1((X_1^1, X_2^1)^\top, P_1) &\subseteq (X_3^1, X_4^1)^\top, \\
 g^2((X_3^2, X_4^2)^\top, P_2) &\subseteq X_2^2, \\
 X_2^2 &\subseteq X_2^1, \\
 X_3^1 &\subseteq X_3^2, \\
 X_4^1 &\subseteq X_4^2, \\
 X_2^2 &\subseteq X_2^0.
 \end{aligned} \tag{5}$$

The barrier function, h_i^k with weights a_i^k , for navigating the tensions between subcontracts while avoiding extremes is defined for each variable x_i based on its role in subcontract f^k :

$$h_i^k = \begin{cases} 0, & \text{if } x_i \text{ not connected to } f^k, \\ -a_i^k (\ln |\bar{X}_i - \overline{RE}_{x_i}| + \ln |\underline{X}_i - \underline{RE}_{x_i}|), & \text{if } x_i \text{ input of } f^k, \\ -a_i^k (\ln |\bar{X}_i - \overline{RA}_{x_i}| + \ln |\underline{X}_i - \underline{RA}_{x_i}|), & \text{if } x_i \text{ output of } f^k. \end{cases}$$

The optimization then aims to minimize the sum of these barrier functions for all variables across all subcontracts, subject to the constraints defining the feasible space:

$$\begin{aligned}
 &\text{minimize } \sum_k \sum_i h_i^k \\
 &\text{subject to constraints (5).}
 \end{aligned} \tag{6}$$

Optimization through constraint programming ensures that the subcontracts are theoretically robust and practically realizable within the confines of supplier capabilities. It adheres to

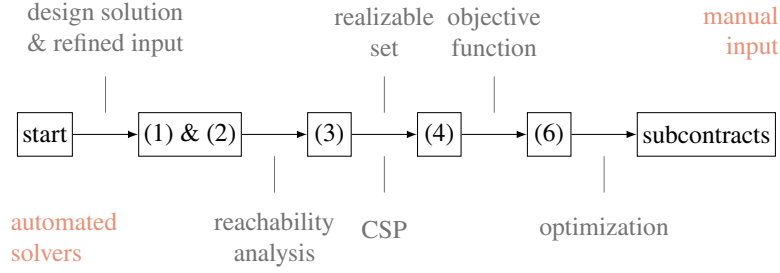


Fig. 6 The overall correct-by-construction scheme for top-to-bottom RE. The operations on top require manual input systematically augmented by the automated outputs from the tools on the bottom.

established correctness criteria and ensures the system’s overall performance and efficiency. The minimization of the cumulative barrier functions across all variables and subcontracts, subject to constraints, underscores the balance between practical feasibility and theoretical robustness in system design. The outcome is a set of subcontracts optimized not only for individual correctness but also for their collective contribution to fulfilling the top-level requirement of the system.

3.5 The semi-automated requirement decomposition process

The semi-automated requirements decomposition for subcontract derivation comprises of multiple levels, some of which are: manual input at the top, a logical sequence organizing the process in the middle, and the formal methods supporting automatic computation at the bottom (fig. 6).

The process initiates with the manual input of the design solution and the refined input set. First, conditions (1) and (2) are automatically checked to verify if the design solution refines the upper-level requirement comprehensively. If condition (1) is not met, a new design solution is sought; similarly, if condition (2) is not met, a new refined input set is manually selected. Once conditions (1) and (2) are satisfied, the lower bound of the subcontracts, indicated by condition (3), is determined using the reachable sets identified in the reachability analysis.

Subsequently, the realizable sets in condition (4) are input to automatically compute the upper bound of the subcontracts through constraint programming. Failing to find a feasible solution indicates conflicts within the realizable sets, implying supplier incompatibility. Similarly, if the contracted realizable sets do not encompass their corresponding reachable sets, some suppliers may be unable to provide the necessary devices for system operation.

Finally, the requirement parameters are recast into an optimization problem to determine the optimal subcontracts. The system automatically derives the constraints from conditions (5). However, the objective function (6) requires manual input based on the designer’s specific goals, a reasonable expectation in the context of RE.

Top down vs. bottom up

Firstly, our method is driven by the top-level requirement, ensuring that every lower-level component directly contributes to satisfying the overall system requirement. This contrasts with bottom-up approaches, such as those based on process algebras or component-based design, which typically start with predefined components and attempt to compose them to meet

system-level goals. Our approach maintains traceability throughout the design process, linking each sub-requirement explicitly to higher-level objectives.

Secondly, we incorporate formal verification at each decomposition step. Unlike purely bottom-up approaches where compatibility issues may only become apparent during integration, our method allows designers to ensure that the proposed decomposition satisfies the higher-level requirement before proceeding further. This early validation is particularly valuable in complex systems where emergent properties may not be evident from individual component specifications alone.

Furthermore, our method supports an iterative refinement process guided by formal and *designer-led* feedback. This allows for adjustments to be made early in the design process, driven by top-level requirements, rather than being constrained by existing component implementations. In contrast, bottom-up approaches often struggle to accommodate changes in system-level requirements, potentially requiring significant re-engineering of component interfaces and interactions.

While our approach begins with a top-down decomposition, it does not preclude the integration of existing components or bottom-up considerations. The formal contracts used in our method provide a rigorous interface between high-level requirements and low-level implementations. This allows designers to incorporate implementation constraints or existing components into the top-down design process, effectively bridging the gap between top-down design intentions and bottom-up implementation realities.

In practice, our method offers a more flexible and comprehensive approach to system design than purely bottom-up methods. By maintaining different levels of abstraction throughout the design process, it allows for the integration of both high-level design concepts and low-level implementation details. This can lead to more coherent system designs and potentially reduce integration issues that often plague bottom-up approaches.

In summary, while the initial decomposition in our method involves manual input from the designer, the overall approach maintains a top-down nature through its requirement-driven process, formal verification at each step, and ability to iteratively refine the design based on system-level requirements. This approach offers distinct advantages over traditional bottom-up methods, particularly in handling complex systems with evolving requirements that are capturable in continuous bounded sets.

4 Cruise control requirements decomposition

To illustrate the semi-automatic requirement decomposition process, we consider the design of a cruise control system. This system maintains the vehicle's speed within specific bounds under various operational conditions. The high-level requirement defines the speed parameters in natural language as follows:

For any initial speed $v(0)$ within 23.0 m s^{-1} to 30.0 m s^{-1} and a reference speed v_{ref} within 34.5 m s^{-1} to 35.5 m s^{-1} , the actual speed $v(t)$ must remain within 20 m s^{-1} to 40 m s^{-1} at all times during a 100 s simulated period. Furthermore, after 20 s, the speed should stabilize within 33.5 m s^{-1} to 36.5 m s^{-1} .

This high-level requirement is deliberately defined with a degree of looseness to facilitate lower-level refinement and adaptation.

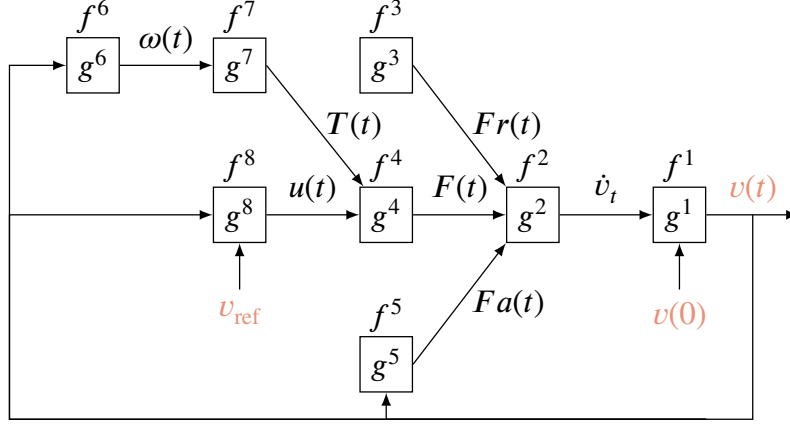


Fig. 7 A candidate design solution for a cruise control system (adapted from Åström and Murray (Åström and Murray, 2010)).

The contract to be refined, denoted as f^0 , is characterized by specific input and output sets that dictate the expected system behavior. For any v_{ref} and $v(0)$ within the input set, the corresponding $v(t)$ must adhere to the defined output set boundaries:

$$f^0 : v_{\text{ref}} \times v(0) \rightarrow v(t)$$

where

$$\begin{aligned} v_{\text{ref}} &= [34.5, 35.5] \text{ m s}^{-1}, \\ v(0) &= [23.0, 30.0] \text{ m s}^{-1}, \text{ and} \\ v(t) &= \begin{cases} [20, 40] \text{ m/s} & \text{for } t \in [0, 100] \text{ s} \\ [33.5, 36.5] \text{ m/s} & \text{for } t \in [20, 100] \text{ s} \end{cases}. \end{aligned}$$

By setting these high-level requirements and defining the input and output sets for f^0 , we lay the groundwork for the subsequent stages of system design and refinement, ensuring that the final implementation aligns with the desired performance criteria and operational constraints.

The design solution for the cruise control system (fig. 7) demonstrates the semi-automatic requirement decomposition process. The objective of this section is not to develop the optimal cruise control system but to use it as a case study for explaining the decomposition process—often, we have to sacrifice optimal behavior for safety. The system is articulated through eight functions ($g^1, g^2, \dots, g^8$) (table 1) and comprises eight variables:

$$(v, \dot{v}, Fr, F, Fa, T, \omega, u)^\top = (x_1, x_2, \dots, x_8)^\top.$$

Key uncertain parameters include the total mass (m) and the maximal engine speed (ω_m). The input sets are extended to accommodate the design solution's refinement of the high-level contract f^0 , with v_{ref} in the range of 34.0 m s^{-1} to 36.0 m s^{-1} and $v(0)$ within 22.0 m s^{-1} to 30.0 m s^{-1} .

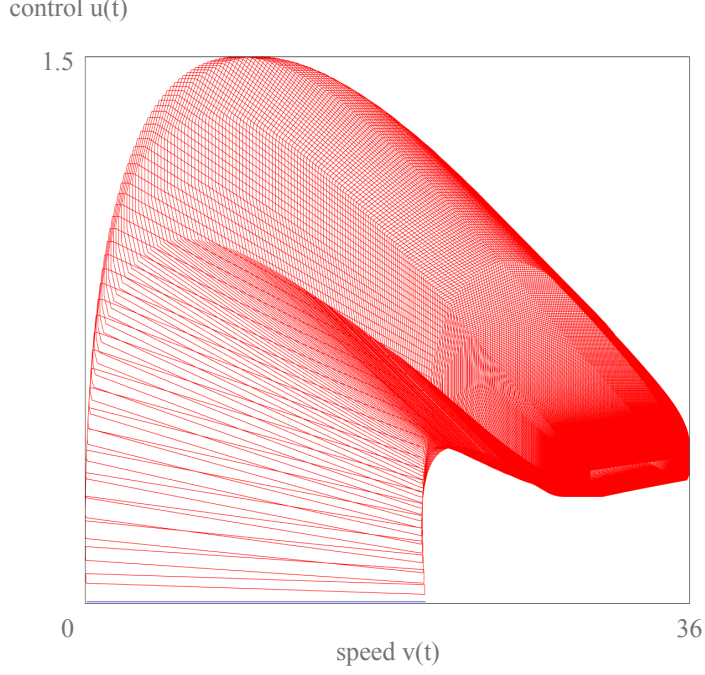


Fig. 8 Evolution of the reachable sets of $(v(t), u(t))^T$ during $t \in [0, 100]$.

The reachable sets for the system variables, indicative of the range of possible states the system can attain over time, are computed using CORA (Althoff, 2015). The evolution of the system state over the simulation period is characterized by a transition from a broad initial state to a progressively more focused and stable region. Initially, the system state starts from a specific set of conditions and evolves, as demonstrated by the progression and narrowing of the reachable sets (figs. 8 and 9).

Specifically, the system begins with a wide range of possible states, which is then gradually confined to a smaller region, reflecting the system's convergence to a stable state. This dynamical behavior is evidenced by a continuous plot showing how the system's state converges over time, beginning with a broad spread and then moving towards a more concentrated area, indicative of system stabilization (fig. 8). Additionally, discrete snapshots of the system's state provide insight into its temporal progression, showing a narrowing of the possible states as time advances (fig. 9). The detailed bounds of the reachable sets for each variable provide insight into the operational limits and capabilities of the system (table 2).

Firstly, the torque $T(t)$ is constrained within 179.939 to 200.000, adhering to the maximum torque limit $T_m = 200$. Similarly, the engine speed $\omega(t)$ is bound within 219.665 to 362.297, respecting the maximum engine speed range of 365 to 450. These constraints ensure that the system's operational parameters remain within the physical capabilities of the vehicle.

Secondly, the design solution's output aligns with the upper-level contract's stipulated requirements. Conditions (1) and (2) are fulfilled, indicating that the design solution effectively refines the high-level contract. Consequently, the reachable sets serve as the lower bounds for the subcontracts, satisfying condition (3). Specifically, for $t \in 0$ s to 100 s, the vehicle's

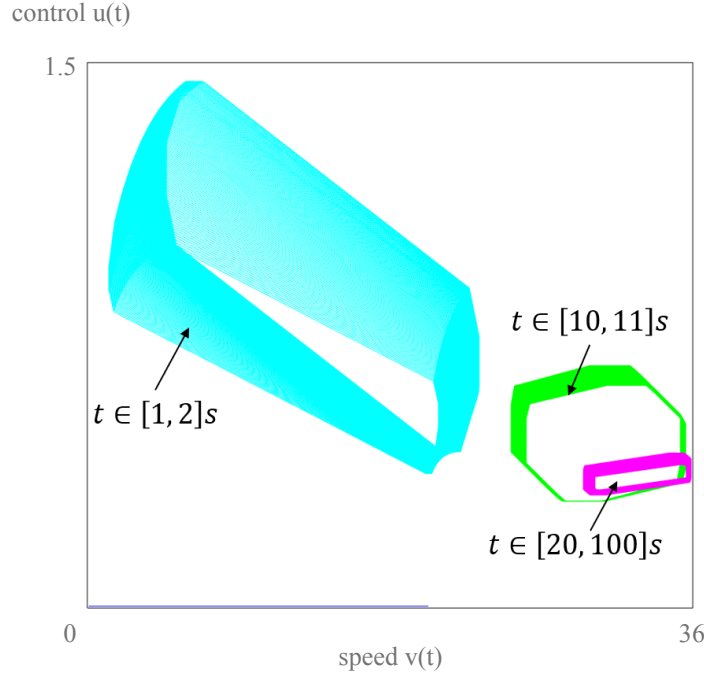


Fig. 9 Discrete sampling of reachable sets $(v(t), u(t))^T$ during $t \in [0, 100]$.

speed $v(t)$ is maintained within 21.966 m s^{-1} to 36.230 m s^{-1} , comfortably within the broader speed range of 20.0 m s^{-1} to 40.0 m s^{-1} . For the latter part of the trajectory, $t \in 20 \text{ s}$ to 100 s , the speed $v(t)$ narrows down to 33.651 m s^{-1} to 36.192 m s^{-1} , aligning with the more stringent requirement of 33.5 m s^{-1} to 36.5 m s^{-1} .

4.1 Realizable sets

The formulation of the design solution into a CSP yields three possible outcomes.

1. Successful contraction of the original sets encompassing the lower bounds, ensuring that all necessary operational parameters are included.
2. Successful contraction of the original sets but excluding some lower bounds might indicate the need for further refinement or adjustment in the system's parameters or constraints.
3. Failure to contract, suggesting internal conflicts among the original sets. This outcome reveals inconsistencies or infeasibilities within the system design that require resolution.

Various test cases are presented (table 3) to illustrate different possible outcomes from the CSP formulation, assuming consistency in the original realizable sets across variables for generality. Test 1 demonstrates the successful contraction of the original sets, with the resulting sets exceeding the lower bounds and qualifying as upper bounds for the subcontracts.

In Test 2, the torque's realizable set $T(t)$ is modified to 0 to 180. This adjustment results in contracted sets of $v(t)$, $\omega(t)$, $T(t)$, and $Fa(t)$ that do not cover the respective lower bounds. This suggests a potential need for a device capable of providing a broader torque range.

Table 1 The functions and parameters of the design solution.

g^i	Function	Explanation
$x_1 = g^1(x_2)$	$v(t) = \int_0^t \dot{v} dt + v(0)$	The definition of speed change
$x_2 = g^2(x_3, x_4, x_5, m)$	$\dot{v}(t) = (F(t) - Fr - Fa(t))/m$	The driving force $F(t)$, the aerodynamic drag $Fa(t)$ and the tyre friction Fr are the force considered.
$x_3 = g^3(m)$	$Fr = mgC_r$	The tyre friction, where C_r is coefficient for the tyre friction.
$x_4 = g^4(x_6, x_8)$	$F(t) = \alpha * u(t)T(t)$	The driving force, where α is the gear rate, $u(t)$ is the control input, $T(t)$ is the torque.
$x_5 = g^5(x_1)$	$Fa(t) = \frac{1}{2}\rho C_d A v(t)^2$	The aerodynamic drag, where ρ is the air density, C_d is the shape-dependent aerodynamic drag coefficient and A is the frontal area of the car.
$x_6 = g^6(x_7, \omega_m)$	$T(t) = T_m(1 - \beta(\frac{\omega(t)}{\omega_m} - 1)^2)$	The torque is correlated to the engine speed $\omega(t)$ (RPM) and the maximum torque T_m is obtained at engine speed ω_m .
$x_7 = g^7(x_1)$	$\omega(t) = \alpha * v(t)$	The engine speed (RPM) is correlated with the car speed.
$x_8 = g^8(x_1)$	$u(t) = p(v_{\text{ref}} - v(t)) + i \int_0^t (v_{\text{ref}} - v(t))dt$	The PI controller.
$x_1 = v, x_2 = \dot{v}, x_3 = Fr, x_4 = F, x_5 = Fa, x_6 = T, x_7 = \omega, x_8 = u$		
$m \in [900, 1100], \omega_m \in [365, 450], v_{\text{ref}} \in [34.0, 36.0], v(0) \in [22.0, 30.0], \alpha = 10, g = 9.8, C_r = 0.01, C_d = 0.32, \rho = 1.3, A = 2.4, \beta = 0.4, T_m = 200, p = 0.1, i = 0.5$		

Test 3 modifies the aerodynamic drag $Fa(t)$ to a range of 600 to 1000 compared to Test 2. This change yields no feasible contracted realizable sets, indicating a significant internal conflict. Notably, the contracted set of $Fa(t)$ in Test 2 is 0 to 543.6, which has no overlap with the original set of $Fa(t)$ in Test 3 (600 to 1000), leading to the conflicts observed.

Table 2 The reachable sets during $t \in [0, 100]$ seconds.

Variable	Reachable set
$x_1 = v(t)$	[21.96649, 36.22973]
	[33.65083, 36.19185] for $t > 20$ s
$x_2 = \dot{v}(t)$	[-0.62937, 2.89036]
$x_3 = Fr(t)$	[88.20000, 107.80000]
$x_4 = F(t)$	[-9.33497, 2997.78280]
$x_5 = Fa(t)$	[240.87740, 655.24649]
$x_6 = T(t)$	[179.93903, 200.00000]
$x_7 = \omega(t)$	[219.66494, 362.29727]
$x_8 = u(t)$	[-0.00472, 1.50660]

The contracted realizable sets from Test 1 are adopted as the upper bounds for the subcontracts, satisfying condition (4).

Formulating the design solution into a CSP is a formal approach to refining and validating system requirements and designs. It categorically addresses the system's feasibility through three possible outcomes, each offering specific insights into its design quality. Successful contraction ensures all operational parameters are included, indicating a robust design. Contraction excluding some lower bounds signals a need for further refinement, prompting engineers to reassess and tweak system parameters or constraints. Failure to contract points out internal conflicts, urging a thorough reevaluation of the system design to resolve inconsistencies or infeasibilities. The test cases exemplify how different scenarios affect the design's validity, guiding engineers to make informed decisions to enhance system performance and reliability. This systematic approach in CSP enables a clear, step-by-step refinement process, ensuring that the system meets its intended requirements and operates effectively within its defined parameters.

Table 3 Test cases for the realizable sets.

	Test 1	Test 2	Test 3
Variable	Original realizable sets		
$x_1 = v(t)$	[0,50]	[0,50]	[0,50]
$x_2 = v(\dot{t})$	[-1.5,3]	[-1.5,3]	[-1.5,3]
$x_3 = Fr(t)$	[70,120]	[70,120]	[70,120]
$x_4 = F(t)$	[-250,3500]	[-250,3500]	[-250,3500]
$x_5 = Fa(t)$	[0,1000]	[0,1000]	[600, 1000]
$x_6 = T(t)$	[0,250]	[0,180]	[0,180]
$x_7 = \omega(t)$	[0,450]	[0,450]	[0,450]
$x_8 = u(t)$	[-0.5,2]	[-0.5,2]	[-0.5,2]
Variable	Contracted realizable sets		
$x_1 = v(t)$	[0, 45.000]	[0, 33.000]	
$x_2 = v(\dot{t})$	[-1.5, 3.0]	[-1.0, 3.0]	
$x_3 = Fr(t)$	[88.2,107.8]	[88.2, 107.8]	
$x_4 = F(t)$	[-250.0, 3500.0]	[-250,3500.0]	Empty
$x_5 = Fa(t)$	[0,1000.0]	[0, 543.6]	
$x_6 = T(t)$	[120.0, 220.0]	[120.0, 180.0]	
$x_7 = \omega(t)$	[0,450.0]	[0,330.0]	
$x_8 = u(t)$	[-0.208, 2.0]	[-0.208, 2.0]	

Table 4 Optimization problem formulation for achieving optimal subcontracts.

Decision variables	$\underline{X}_i$ and $\overline{X}_i$ where $i = 1, 2, \dots, 8$
Objective function	$minimize \sum_{k=1}^8 \sum_{i=1}^8 h_i^k$
Constraints	$\overline{RA}_{x_i} \leq \overline{X}_i \leq \overline{RE}_{x_i}$ $\overline{RE}_{x_i} \leq \underline{X}_i \leq \underline{RA}_{x_i}$ $\underline{X}_1 \geq 20, \overline{X}_1 \leq 40$ $\underline{X}_7 \leq g^7(\underline{X}_1)$ $\overline{X}_7 \geq g^7(\overline{X}_1)$ $\underline{X}_6 \leq g^6(\underline{X}_7, \overline{\omega}_m)$ $\overline{X}_6 \geq g^6(\overline{X}_7, \overline{\omega}_m)$ $\underline{X}_5 \leq g^5(\underline{X}_1)$ $\overline{X}_5 \geq g^5(\overline{X}_1)$ $\underline{X}_4 \leq g^4(\underline{X}_6, \underline{X}_8)$ $\overline{X}_4 \geq g^4(\overline{X}_6, \overline{X}_8)$ $\underline{X}_2 \leq g^2(\underline{X}_4, \overline{X}_3, \overline{X}_5, \overline{m})$ $\overline{X}_2 \geq g^2(\overline{X}_4, \underline{X}_3, \underline{X}_5, \underline{m})$ $\underline{\omega}_m = 365, \overline{\omega}_m = 450, \underline{m} = 900, \overline{m} = 1100$

4.2 Trade-off study: optimal subcontracts

We formulate the optimization problem to determine the optimal subcontracts for the cruise control example (table 4). The decision variables, $\underline{X}_i$ and $\overline{X}_i$, represent the lower and upper bounds of subcontract X_i for $i = 1, 2, \dots, 8$. The objective function is tailored to the designer's preference and is constructed following condition (6). The parameters required to calculate h_i^k are provided in table 5.

The constraints for the optimization are established according to (6), where $x = (x_1, x_2, \dots, x_8)^\top$. The reachable sets $\overline{RA}_{x_i}$ and realizable sets $\overline{RE}_{x_i}$ are documented in table 5, and the functions g^i are outlined in table 1.

The optimal subcontracts from the formulation are depicted in table 6. The given contract at level 0 is refined by the specifications in the second row, while the decomposed requirements, or the subcontracts, are presented in the third row (table 6).

Control input u is included at level 0, and notably, f^1 and f^8 are absent from the level 1 subcontracts. This is because speed v and control input u are system-wide properties and cannot be relegated to lower levels. Specifically, for f^1 , reasoning about $v(t)$ from $v(t)$ requires knowledge of the entire system's behavior over time, which is determined by the composition of the whole system. However, at level 0, $v(t)$ can be contextualized with the inputs $v(0)$ and v_{ref} . This relationship implies that $v(t)$ is inherently associated with the highest level of system abstraction and must be ensured at level 0. Accordingly, g^1 and g^8 do not form part of the optimization constraints (table 4).

Table 5 The parameters for the trade-off study.

X_i	$[RA_{x_i}, \overline{RA_{x_i}}]$	$[RE_{x_i}, \overline{RE_{x_i}}]$	Difficulty coefficients
X_1	[22.00, 36.23]	[0, 45]	$a_1^1 = 0.9, a_1^5 = 0.5,$ $a_1^7 = 0.1, a_1^8 = 0.8$
X_2	[-0.63, 2.89]	[-1.5, 3.0]	$a_2^1 = 0.6, a_2^2 = 0.6$
X_3	[88.20, 107.80]	[88.2, 107.8]	$a_3^3 = 0.5, a_3^2 = 0.4$
X_4	[-9.36, 2997.78]	[-250.0, 3500.0]	$a_4^4 = 0.3, a_4^2 = 0.8$
X_5	[240.88, 655.25]	[0, 1000.0]	$a_5^5 = 0.4, a_5^2 = 0.7$
X_6	[179.94, 200.00]	[120.0, 220.0]	$a_6^6 = 0.1, a_6^2 = 0.9$
X_7	[219.67, 362.30]	[0, 450.0]	$a_7^7 = 0.5, a_7^6 = 0.6$
X_8	[-0.005, 1.51]	[-0.29, 2.0]	$a_8^8 = 0.9, a_8^4 = 0.9$

By defining explicit upper and lower bounds for each subcontract, the framework precisely delineates each component's operational space, directly linking the technical specifications to the system's overall objectives. This precision in specifying requirements ensures that each subsystem or component meets its individual objectives and aligns with the overarching system goals, enhancing the coherence and efficiency of the system as a whole. The optimization process iteratively refines these subcontracts, allowing for a calibration of requirements responsive to internal system constraints and external performance criteria.

4.3 Discussion

This paper has introduced a formalized approach for semi-automatic requirement decomposition based on CBD. This research augments a top-down derivation of sub-requirements common in systems engineering with formal methods typically used in bottom-up compositional verification, offering a more systematic and intuitive way to bridge high-level system goals with detailed technical specifications. The formalism developed ensures mathematical correctness and provides a structured method for refining system requirements, further enhanced through integrating formal methods in reachability analysis and CSP.

The results from applying this methodology, notably demonstrated through the cruise control system case study, underscore its practicality. The approach not only streamlines the process of requirement decomposition but also enhances the clarity and reliability of the resulting system specifications. However, the critical insight gleaned from this work is

identifying a significant gap in traditional RE practices: the need for a systematic, top-down approach that can effectively handle modern systems' complexity and multi-level nature. The proposed approach fills this gap by offering a structured and formalized method that aligns with the nuanced demands of system design.

Furthermore, the case study validates the possible depth of analysis and refinement. Yet, it also brings to light the challenges and limitations associated with modeling requirements as continuous bounded sets, especially for systems that might be more accurately represented as transition systems dealing with discrete traces. This recognition opens further research and development avenues, emphasizing the need to extend and adapt the methodology to a broader range of system types and complexities. The ultimate aim is to create a robust toolkit for RE that can adapt to the evolving system design landscape, thereby enhancing system development's efficiency, reliability, and traceability in the RE field.

Despite the current methodology's emphasis on continuous bounded sets, its core principles and formal approach retain their applicability to transition systems. The transition system, a model dealing with discrete states and events, requires a nuanced treatment of requirements and sub-requirements that are inherently discrete and often complex. While the current approach may necessitate adjustments and extensions to fully encapsulate these systems' discrete nature, the foundational goal of ensuring a mathematically correct, top-down decomposition of requirements remains unchanged. By incorporating concepts and tools suited to discrete systems, such as state-transition matrices, automata theory, or model-checking algorithms, the methodology can be adapted to address the unique challenges and characteristics of transition systems. This adaptation would further demonstrate the methodology's potential as a comprehensive toolkit in RE, capable of accommodating the diverse modeling needs of modern complex systems, thereby maintaining the rigor and structured approach central to enhancing system design and development processes.

By recognizing the limitations of current practices, particularly in handling diverse and complex systems, this research advocates for a more adaptable, robust approach. Incorporating formal methods, such as reachability analysis and CSP, into RE enhances precision and rigor, ensuring that each component's specifications are theoretically sound and practically viable. The emphasis on a structured, top-down approach reflects an understanding of the relationships and dependencies within modern systems, advocating for a strategy that is both comprehensive and flexible. As this methodology continues to evolve and adapt, it can provide practitioners with a powerful tool to navigate the complexities of system design with greater confidence and efficacy. Pursuing a more adaptable, inclusive approach is not just an academic exercise but a strategic imperative to keep pace with the rapid technological advancements and the increasing sophistication of system designs, particularly in cyber-physical systems.

A note on requirement capture and computational tools for practitioners

This case study was conducted using MATLAB to perform the computational aspects of the methodology. While natural language requirements were manually translated within MATLAB for illustrative purposes, the practicality of the approach can be enhanced by leveraging existing toolchains. For *formal requirements capture*, structured methods such as the Easy Approach to Requirements Syntax (EARS) (Mavin et al, 2009) are widely used in RE and could serve as a valuable intermediary layer between natural language specifications and contract theory as a *requirements formalism*. *Reachability analysis* was performed using CORA, a common choice

Table 6 The correct-by-construction optimal subcontracts.

	Input	Output
Level 0 contract	$v_{\text{ref}} \in [34.5, 35.5]$ $v(0) \in [22.0, 30.0]$	$v \in [20, 40]$
Level 0 refinement	$v_{\text{ref}} \in [34.0, 36.0]$ $v(0) \in [22.0, 30.0]$	$v \in [21.8, 38.4]$ $u \in [-0.1, 1.511]$
	f^2	$Fr \in [88.2, 107.8]$ $F \in [-159.1, 3024.0]$ $F_a \in [237.6, 827.6]$ $\dot{v} \in [-1.1, 3]$
	f^3	NA $Fr \in [88.2, 107.8]$
Level 1 subcontract	f^4	$T \in [150, 200.1]$ $u \in [-0.1, 1.511]$ $F \in [-159.1, 3024.0]$
	f^5	$v \in [21.8, 38.4]$ $F_a \in [237.6, 827.6]$
	f^6	$\omega \in [109.8, 406.1]$ $T \in [150, 200.1]$
	f^7	$v \in [21.8, 38.4]$ $\omega \in [109.8, 406.1]$

for continuous systems. However, alternative tools exist for different use cases. For instance, SpaceX (Frehse et al, 2011) is well-suited for hybrid systems, addressing both continuous and discrete behaviors. *Constraint solving and optimization*, can be effectively handled by established tools such as Google OR-Tools or IBM CPLEX, depending on the specific problem formulation and complexity. To facilitate practical implementation, *integrating the development environment* is crucial. Options include expanding the MATLAB environment with custom requirements capture functionality, or developing integrated solutions using platforms like Eclipse and Jupyter notebooks. These integrated environments would streamline the workflow from requirement specification to analysis and optimization.

5 Related Work

One of the issues in introducing design errors in embedded systems is due to a lack of formalism in translating system requirements into subsystem requirements and subsequently into code (Salter, 1976). In particular, Moore supported this observation by highlighting that existing formal techniques predominantly focus on the bottom-up composition of component-level

specifications into system-level specifications rather than on a top-down derivation of component requirements from higher-level system requirements (Moore, 1990). Despite the passage of time, the literature continues to be skewed towards bottom-up formalisms and component-based composition, with scant attention to top-down decomposition. This paper aims to address this gap by concentrating on top-down requirement decomposition.

Industry standards delineate system and software requirement document structure (IEEE/ISO/IEC 29148-2018, 2018). Arora et al. introduced the concept of requirement boilerplates to confine the syntax of requirement sentences to predefined linguistic structures, significantly reducing ambiguity and enhancing the feasibility of automated analysis (Arora et al, 2014). Mahendra and Ghazarian conducted extensive studies on requirements patterns to aid practitioners in identifying, analyzing, and structuring software system requirements (Mahendra and Ghazarian, 2014). Further, Reinkemeier et al. developed a pattern-based requirement specification language specifically targeting timing requirements in the automotive domain (Reinkemeier et al, 2011). Project CESAR adopts a formalism that translates requirements from natural language into boilerplates and pattern-based requirements for automated analysis (Jolliffe, 2010). While these formalisms contribute to reducing ambiguity in requirement expression, they do not incorporate formal approaches for decomposition.

In component-based composition, the architecture analysis & design language (AADL) is recognized as a leading architecture description language, offering formal modeling concepts for describing and analyzing system architecture through components and their interactions (Feiler et al, 2006). Bozzano et al. developed a model-checker for the functional verification and safety analysis of AADL-based models (Bozzano et al, 2010). Hu et al. presented a methodology for translating AADL into a timed abstract state machine to verify both functional and nonfunctional properties of AADL models (Hu et al, 2015). Johnsen et al. introduced a formal verification technique for automated fault avoidance in systems specified in AADL (Johnsen et al, 2012). Despite AADL’s support for step-wise refinement and architectural verification, it does not facilitate the top-down automatic generation of sub-requirements. Other frameworks such as EAST-ADL, Rodin, and FDR contribute to the landscape (Blom et al, 2016; Abrial et al, 2010; Gibson-Robinson et al, 2014).

Recently, CBD has emerged as an increasingly influential paradigm (Benveniste et al, 2012a). One of its aims is to guide the initial specification process in a precise manner and assess quality-related properties using computational tools (Benveniste et al, 2012b). The theory behind CBD has seen application across various domains. For instance, contract formalism has been utilized in the control design of hybrid systems, virtual testing, architecture design, and the development of property-based proof systems (Sangiovanni-Vincentelli et al, 2012; Benvenuti et al, 2008; Damm et al, 2011; Cimatti and Tonetta, 2012). Tools designed for CBD, like AGREE, MICA, and OCRA, have been instrumental in advancing the RE field (Cofer et al, 2012; Raclet et al, 2011; Cimatti et al, 2013).

Assume-guarantee reasoning has been a powerful technique in CBD for decomposing verification tasks of complex systems into subtasks on individual components. Nuzzo et al (2019) present an assume-guarantee contract framework for cyber-physical system design under probabilistic requirements specified in Stochastic Signal Temporal Logic (StSTL). Chatterjee and Henzinger (2007) define and study the co-synthesis problem, formulating it as an assume-guarantee synthesis problem to synthesize strategies that guarantee desired specifications. Henzinger et al (2002) present an assume-guarantee rule for checking simulation relations

between state transition systems, enabling decomposition of simulation checks. [Henzinger et al \(2001\)](#) introduce a model for hierarchical system design supporting an assume-guarantee principle for mixed parallel-serial contexts, suitable for modeling embedded systems interacting with real-world environments. [McMillan \(1999\)](#) extend circular compositional reasoning to handle liveness properties using a new rule implemented in the SMV proof assistant.

Nonetheless, these methods and tools are predominantly geared towards verification (bottom-up composition) and do not provide a methodology to generate lower-level subcontracts from upper-level contracts precisely and systematically. As highlighted in the contracts literature, transitioning from high-level contracts to an architecture of sub-contracts remains a challenging task often done manually ([Benveniste et al, 2015a](#)). This paper responds to this challenge through semi-automated systems engineering holistically as opposed to other works that focus on synthesizing contracts for a specific instance of component ([dos Santos et al, 2019](#); [Dragomir et al, 2017](#))—which we could incorporate as individual improvements when relevant within the design steps (fig. 6).

6 Conclusion

In this paper, we have developed a formalized approach for semi-automatic requirement decomposition explicitly addressing a critical gap in RE. Our framework introduces a structured perspective in system design by shifting focus from the prevalent bottom-up compositional verification to a systematic top-down derivation of sub-requirements. The formalism established for requirement definition and decomposition ensures correctness and facilitates a systematic refinement of system requirements. Integrating formal methods such as reachability analysis and CSP significantly enhances the automation and precision of the process, aligning with the practical needs and complexities of modern RE. Correct-by-construction commits to continuous refinement, ensuring systems are built to meet current specifications and adapt to future changes and challenges.

We must point out that this correct-by-construction decomposition does not work for any arbitrary requirement but only for ones we can quantify—otherwise understood as *physical* requirements. However, the central insight from this work is that a structured, formal approach to top-down requirement decomposition for physical requirements bridges the gap between high-level system goals and detailed technical specifications. This not only aids in maintaining the integrity and traceability of system requirements and provides a precise roadmap for system architects and engineers. The case study on a cruise control system validates the approach, demonstrating its potential to simplify and clarify the RE process. As systems grow increasingly complex and integrated, the correct-by-construction offers a valuable tool for navigating requirement specification and decomposition, promising to enhance the efficiency and reliability of system design in RE by integrating formal methods early in the system’s lifecycle as a form of “what if” analysis.

References

Abbasipour M (2018) A framework for requirements decomposition, SLA management and dynamic system reconfiguration. PhD thesis, Concordia University

- Abrial J, Butler MJ, Hallerstede S, et al (2010) Rodin: an open toolset for modelling and reasoning in Event-B. *Int J Softw Tools Technol Transf* <https://doi.org/10.1007/S10009-010-0145-Y>
- Althoff M (2015) An introduction to CORA 2015. In: ARCH, <https://doi.org/10.29007/zbkv>
- ANSI/EIA 632 (2003) Processes for engineering a system. Standard, Electronic Industries Alliance
- Arora C, Sabetzadeh M, Briand LC, et al (2014) Requirement boilerplates: Transition from manually-enforced to automatically-verifiable natural language patterns. In: RePa, <https://doi.org/10.1109/REPA.2014.6894837>
- ARP4754B (2023) Guidelines for development of civil aircraft and systems. Tech. rep., SAE
- Aström KJ, Murray RM (2010) Feedback systems: An introduction for scientists and engineers. Princeton University Press
- Baier C, Katoen J (2008) Principles of model checking. MIT Press
- Benveniste A, Caillaud B, Nickovic D, et al (2012a) Contracts for system design. Foundations and Trends in Electronic Design Automation <https://doi.org/10.1561/10000000053>
- Benveniste A, Damm W, Sangiovanni-Vincentelli A, et al (2012b) Contracts for the design of embedded systems part I: Methodology and use cases. Tech. rep., Inria
- Benveniste A, Caillaud B, Nickovic D, et al (2015a) Contracts for systems design: methodology and application cases. Tech. rep., Inria
- Benveniste A, Caillaud B, Nickovic D, et al (2015b) Contracts for systems design: Theory. Foundations and Trends® in Electronic Design Automation <https://doi.org/10.1561/10000000053>
- Benvenuti L, Ferrari A, Mangeruca L, et al (2008) A contract-based formalism for the specification of heterogeneous systems. In: FDL, <https://doi.org/10.1109/FDL.2008.4641436>
- Blom H, Chen D, Kaijser H, et al (2016) EAST-ADL: an architecture description language for automotive software-intensive systems in the light of recent use and research. <https://doi.org/10.4018/IJSDA.2016070101>
- Bozzano M, Cimatti A, Katoen J, et al (2010) A model checker for AADL. In: CAV, https://doi.org/10.1007/978-3-642-14295-6_48
- Braun P, Broy M, Houdek F, et al (2014) Guiding requirements engineering for software-intensive embedded systems in the automotive industry. *Comput Sci Res Dev* <https://doi.org/10.1007/S00450-010-0136-Y>

- Chatterjee K, Henzinger TA (2007) Assume-guarantee synthesis. In: Tools and Algorithms for the Construction and Analysis of Systems
- Cimatti A, Tonetta S (2012) A property-based proof system for contract-based design. In: Cortellessa V, Muccini H, Demirörs O (eds) SEAA, <https://doi.org/10.1109/SEAA.2012.68>
- Cimatti A, Dorigatti M, Tonetta S (2013) OCRA: A tool for checking the refinement of temporal contracts. In: ASE, <https://doi.org/10.1109/ASE.2013.6693137>
- Cofer DD, Gacek A, Miller SP, et al (2012) Compositional verification of architectural models. In: NFM, https://doi.org/10.1007/978-3-642-28891-3_13
- Damm W, Hungar H, Josko B, et al (2011) Using contract-based component specifications for virtual integration testing and architecture design. In: DATE, <https://doi.org/10.1109/DATE.2011.5763167>
- DOT/FAA/AR-08/32 (2009) Requirements engineering management handbook. Tech. rep., U.S. Department of Transportation Federal Aviation Administration
- Dragomir I, Ober I, Percebois C (2017) Contract-based modeling and verification of timed safety requirements within sysml. *Softw Syst Model* <https://doi.org/10.1007/S10270-015-0481-1>
- Feiler PH, Gluch DP, Hudak JJ (2006) The architecture analysis & design language (AADL): An introduction. Tech. rep., CMU Software Engineering Institute
- Filippidis I, Murray RM (2018) Layering assume-guarantee contracts for hierarchical system design. *Proc IEEE* <https://doi.org/10.1109/JPROC.2018.2834926>
- Frehse G, Guernic CL, Donzé A, et al (2011) SpaceEx: Scalable verification of hybrid systems. In: CAV, *Lecture Notes in Computer Science*, https://doi.org/10.1007/978-3-642-22110-1_30
- Gibson-Robinson T, Armstrong PJ, Boulgakov A, et al (2014) FDR3 - A modern refinement checker for CSP. In: TACAS, https://doi.org/10.1007/978-3-642-54862-8_13
- Greising D, Johnsson J (2007) Behind Boeing's 787 delays. *Chicago Tribune*. <https://web.archive.org/web/20231224040320/https://www.buffalo.edu/content/dam/www/news/imported/pdf/December07/ChicagoTribPritchardBoeing.pdf>
- Henzinger TA, Minea M, Prabhu V (2001) Assume-guarantee reasoning for hierarchical hybrid systems. In: HSCC, Berlin, Heidelberg, https://doi.org/10.1007/3-540-45351-2_24
- Henzinger TA, Qadeer S, Rajamani SK, et al (2002) An assume-guarantee rule for checking simulation. *ACM Trans Program Lang Syst* <https://doi.org/10.1145/509705.509707>
- Hu K, Zhang T, Yang Z, et al (2015) Exploring AADL verification tool through model transformation. *J Syst Archit* <https://doi.org/10.1016/J.SYSARC.2015.02.003>

- Iber J, Höller A, Rauter T, et al (2015) Towards a generic modeling language for contract-based design. In: ModComp@MoDELS, URL <https://ceur-ws.org/Vol-1463/paper4.pdf>
- IEEE/ISO/IEC 29148-2018 (2018) Systems and software engineering – life cycle processes – requirements engineering. Standard, C/S2ESC
- Johnsen A, Lundqvist K, Pettersson P, et al (2012) Automated verification of aadl-specifications using UPPAAL. In: HASE, <https://doi.org/10.1109/HASE.2012.22>
- Jolliffe G (2010) Cost-efficient methods and processes for safety relevant embedded systems (CESAR) - an objective overview. In: SSS, https://doi.org/10.1007/978-1-84996-086-1_3
- Kirkman DP (1998) Requirement decomposition and traceability. Requirements Engineering <https://doi.org/10.1007/BF02919970>
- Komoto H, Tomiyama T (2011) A theory of decomposition in system architecting. In: ICED
- Kusiak A, Larson N (1995) Decomposition and representation methods in mechanical design. J Mech Des <https://doi.org/10.1115/1.2836453>
- Le Lann G (1997) An analysis of the Ariane 5 flight 501 failure—a system engineering perspective. In: International Conference and Workshop on Engineering of Computer-Based Systems, <https://doi.org/10.1109/ECBS.1997.581900>
- Mahendra P, Ghazarian A (2014) Patterns in the requirements engineering: A survey and analysis study. WSEAS Transaction on Information Science and Application
- Mavin A, Wilkinson P, Harwood ARG, et al (2009) Easy approach to requirements syntax (EARS). In: RE, <https://doi.org/10.1109/RE.2009.9>
- McMillan KL (1999) Circular compositional reasoning about liveness. In: Correct Hardware Design and Verification Methods, https://doi.org/10.1007/3-540-48153-2_30
- Melancon PW (2008) Categorization and representation of functional decomposition by experts. Tech. rep., Naval Postgraduate School
- Mesarovic MD, Macko D, Takahara Y (2000) Theory of hierarchical, multilevel, systems. Elsevier
- Moore AP (1990) The specification and verified decomposition of system requirements using CSP. IEEE Trans Software Eng <https://doi.org/10.1109/32.58782>
- National Aeronautics and Space Administration, Space Science Library (2010) NASA Systems Engineering Handbook. CreateSpace Independent Publishing Platform
- Nuzzo P, Li J, Sangiovanni-Vincentelli AL, et al (2019) Stochastic assume-guarantee contracts for cyber-physical system design. ACM Trans Embed Comput Syst <https://doi.org/10.1145/3243216>

- Ouimet M, Lundqvist K (2007) Formal software verification: Model checking and theorem proving. Tech. rep., MIT
- Penzenstadler B (2011) DeSyRe-decomposition of systems and their requirements. PhD thesis, Technische Universität München
- Raclet J, Badouel É, Benveniste A, et al (2011) A modal interface theory for component-based design. *Fundam Informaticae* <https://doi.org/10.3233/FI-2011-416>
- Reinkemeier P, Stierand I, Rehkop P, et al (2011) A pattern-based requirement specification language: Mapping automotive specific timing requirements. In: SE Workshopband
- Salter KG (1976) A methodology for decomposing system requirements into data processing requirements. In: ICSE
- Sangiovanni-Vincentelli AL, Damm W, Passerone R (2012) Taming Dr. Frankenstein: Contract-based design for cyber-physical systems. *Eur J Control* <https://doi.org/10.3166/EJC.18.217-238>
- dos Santos CAR, Saleh AH, Schrijvers T, et al (2019) CONDEnSe: Contract based design synthesis. In: MODELS. IEEE, <https://doi.org/10.1109/MODELS.2019.00009>
- Sidky AS, Sud RR, Bhatia S, et al (2002) Problem identification and decomposition within the requirements generation process. Tech. rep., Virginia Tech
- Wagner S, Fernández DM, Felderer M, et al (2019) Status quo in requirements engineering: A theory and a global family of surveys. *ACM Trans Softw Eng Methodol* <https://doi.org/10.1145/3306607>
- Ye K, Foster S, Woodcock J (2020) Compositional assume-guarantee reasoning of control law diagrams using UTP. In: Emergence, Complexity and Computation. https://doi.org/10.1007/978-3-030-15792-0_10